

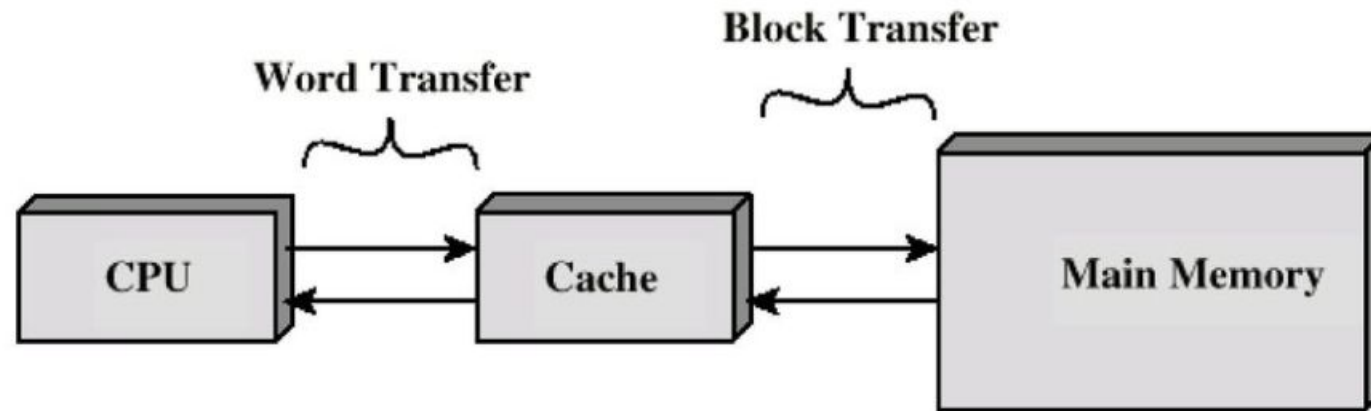
# Cache Memory Mapping

# Hierarchy List

- Registers
- L1 Cache
- L2 Cache
- Main memory
- Disk cache
- Disk
- Optical
- Tape

# Cache

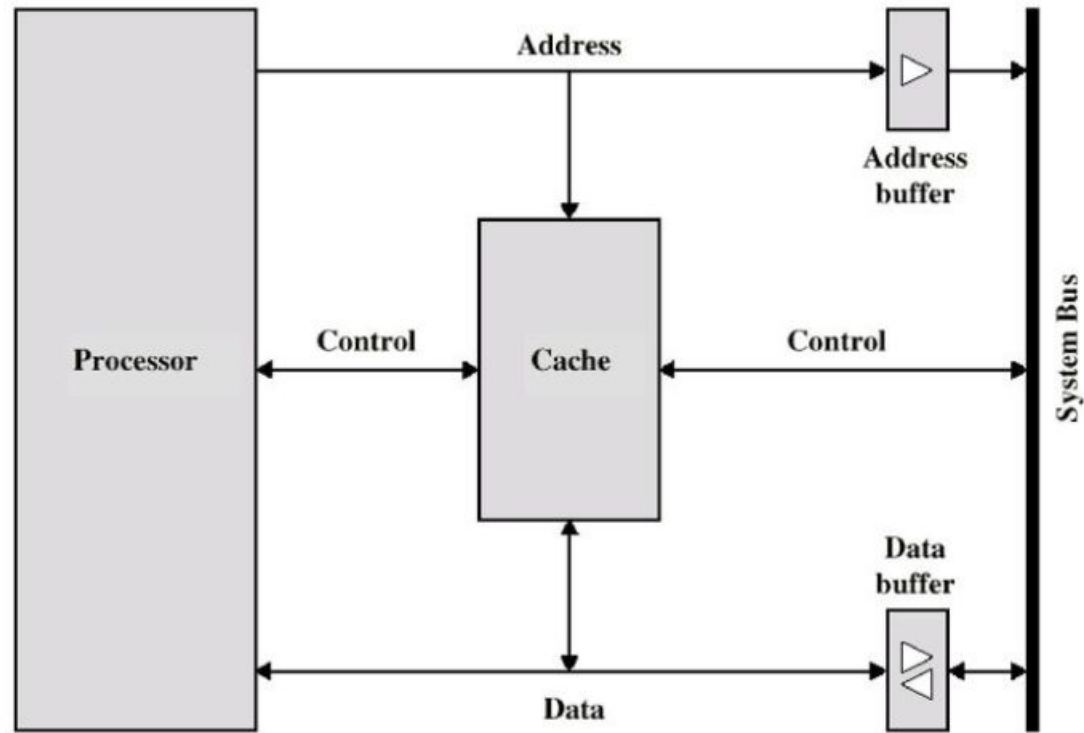
- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module

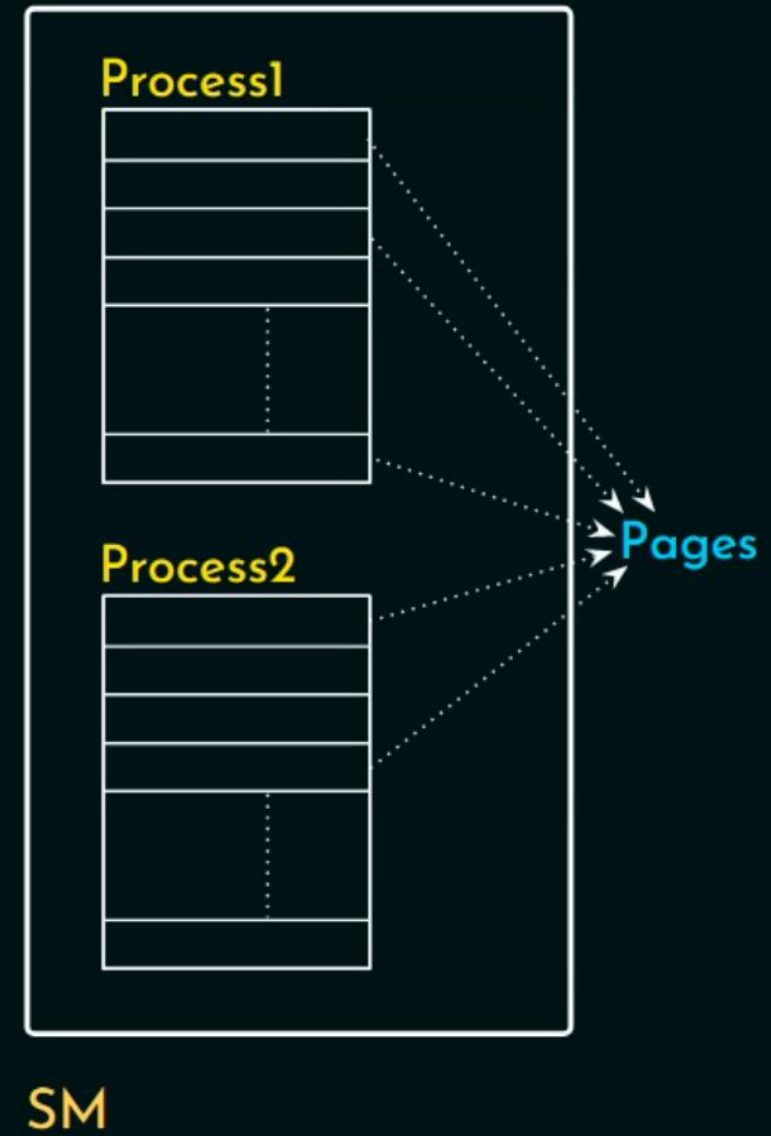
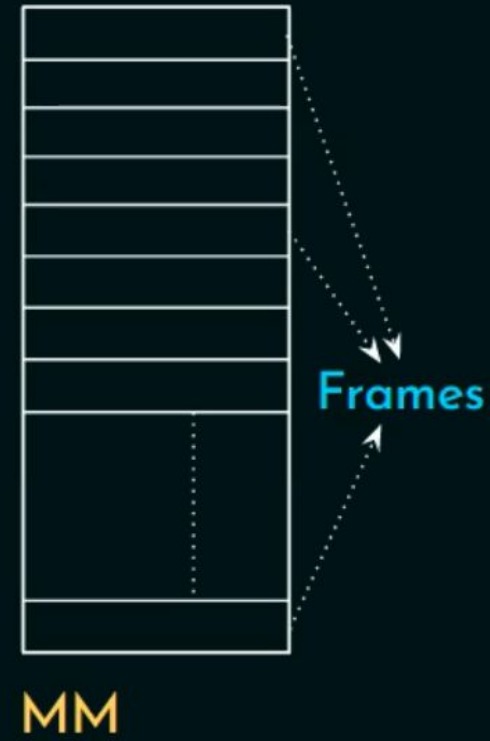
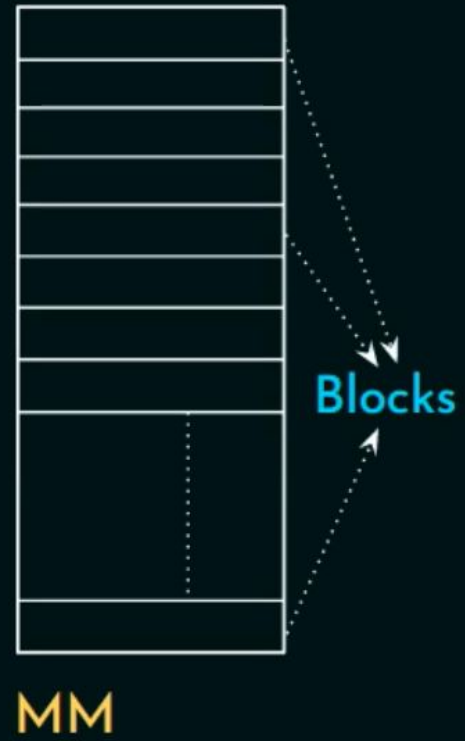
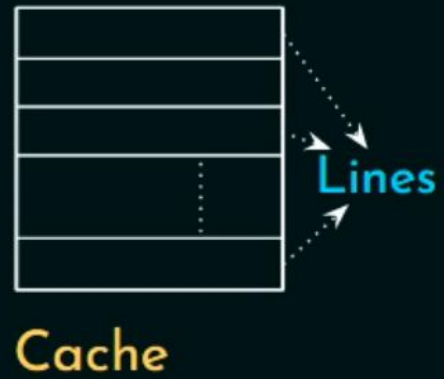


# Cache Design

- Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Block Size
- Number of Caches

# Typical Cache Organization





Line Size = Block Size

Frame Size = Page Size

# Let's see an example

**Main Memory Size** : 64 words

**Block Size** : 4 words

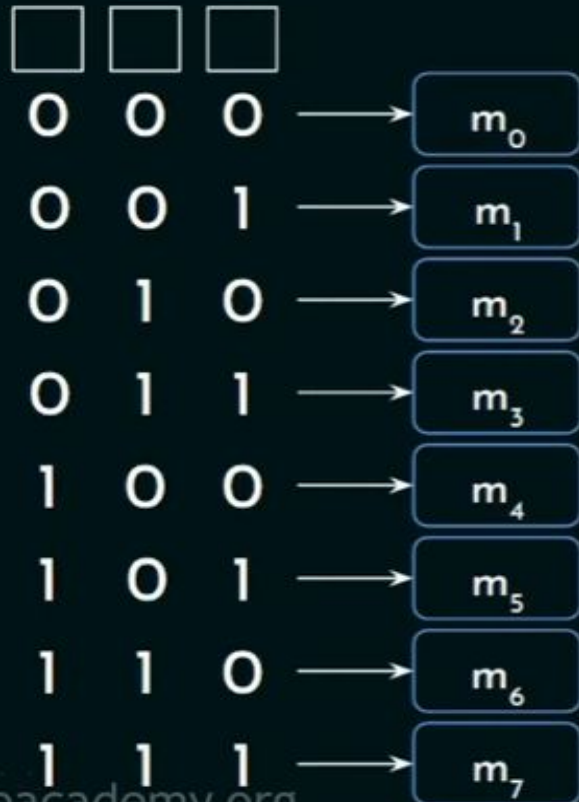
**No. of Blocks in MM** :  $64 / 4 = 16$  i.e. 0, 1, 2, 3, . . . . , 15

|    |  |  |  |  |
|----|--|--|--|--|
| 0  |  |  |  |  |
| 1  |  |  |  |  |
| 2  |  |  |  |  |
| 3  |  |  |  |  |
| 4  |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
|    |  |  |  |  |
| 15 |  |  |  |  |

**Main Memory Size** : 64 words i.e. (0, 1, ..., 63)

**Block Size** : 4 words

**No. of Blocks in MM** :  $64 / 4 = 16$



64 words

$$\log_2 64 = \log_2 2^6 \\ = 6 \text{ bits}$$

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 0  | 1  | 2  | 3  |
| 1  | 4  | 5  | 6  | 7  |
| 2  | 8  | 9  | 10 | 11 |
| 3  | 12 | 13 | 14 | 15 |
| 4  | 16 | 17 | 18 | 19 |
| 5  | 20 | 21 | 22 | 23 |
| 6  | 24 | 25 | 26 | 27 |
| 7  | 28 | 29 | 30 | 31 |
| 8  | 32 | 33 | 34 | 35 |
| 9  | 36 | 37 | 38 | 39 |
| 10 | 40 | 41 | 42 | 43 |
| 11 | 44 | 45 | 46 | 47 |
| 12 | 48 | 49 | 50 | 51 |
| 13 | 52 | 53 | 54 | 55 |
| 14 | 56 | 57 | 58 | 59 |
| 15 | 60 | 61 | 62 | 63 |



# Calculation: Physical Address Bits

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 0  |
| 4  | 5  | 6  | 7  | 1  |
| 8  | 9  | 10 | 11 | 2  |
| 12 | 13 | 14 | 15 | 3  |
| 16 | 17 | 18 | 19 | 4  |
|    |    |    |    |    |
|    |    |    |    |    |
|    |    |    |    |    |
| 60 | 61 | 62 | 63 | 15 |

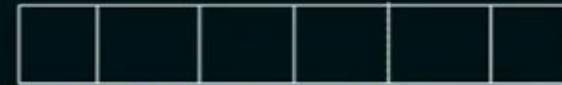
Physical Address Space

16 Blocks

$$\log_2 16 = \log_2 2^4 \\ = 4 \text{ bits}$$

$$\log_2 64 = \log_2 2^6 = 6 \text{ bits}$$

P. A. bits :  
(Physical Address bits)



|   |   |   |                 |
|---|---|---|-----------------|
| 0 | 0 | → | 0 <sup>th</sup> |
| 0 | 1 | → | 1 <sup>st</sup> |
| 1 | 0 | → | 2 <sup>nd</sup> |
| 1 | 1 | → | 3 <sup>rd</sup> |

# Calculation: Block/Line size of Cache

Cache Size : 16 words

Line Size : 4 words

Block Size : 4 words

Block Size = Line Size

No. of Lines in Cache :  $16 / 4 = 4$  i.e. 0, 1, 2, 3

4 Lines

$$\log_2 4 = \log_2 2^2$$

$$= 2 \text{ bits}$$



0 0 → 0

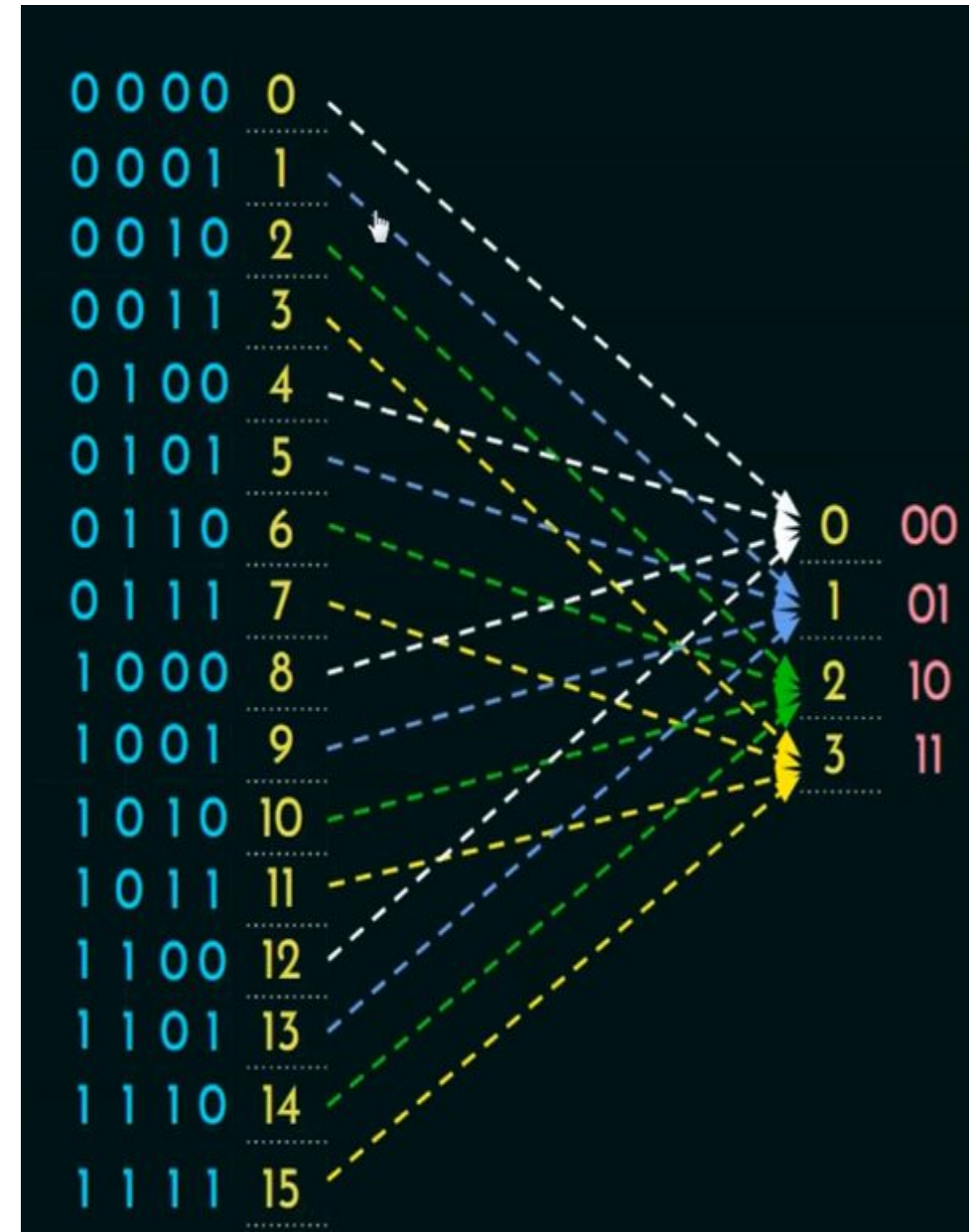
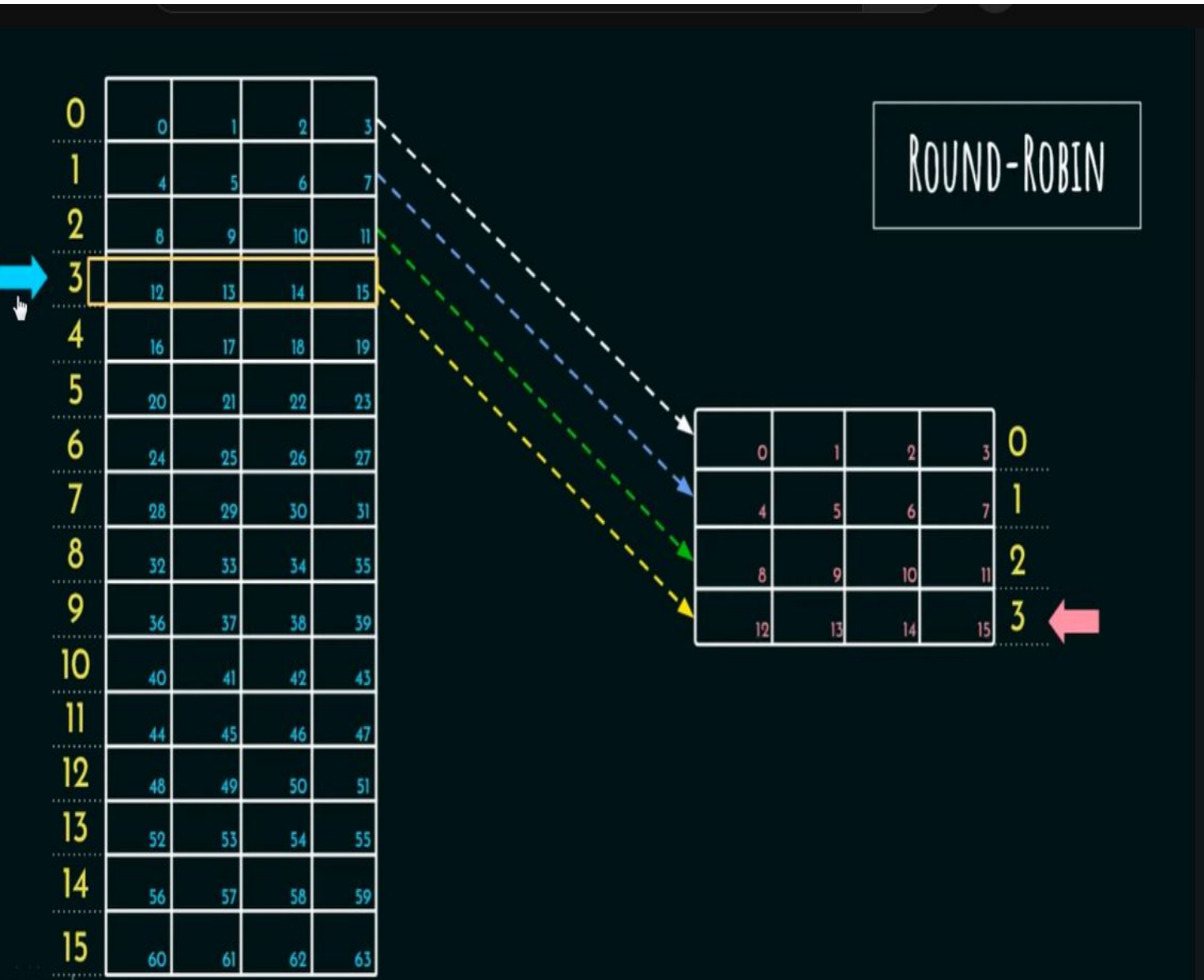
0 1 → 1

1 0 → 2

1 1 → 3

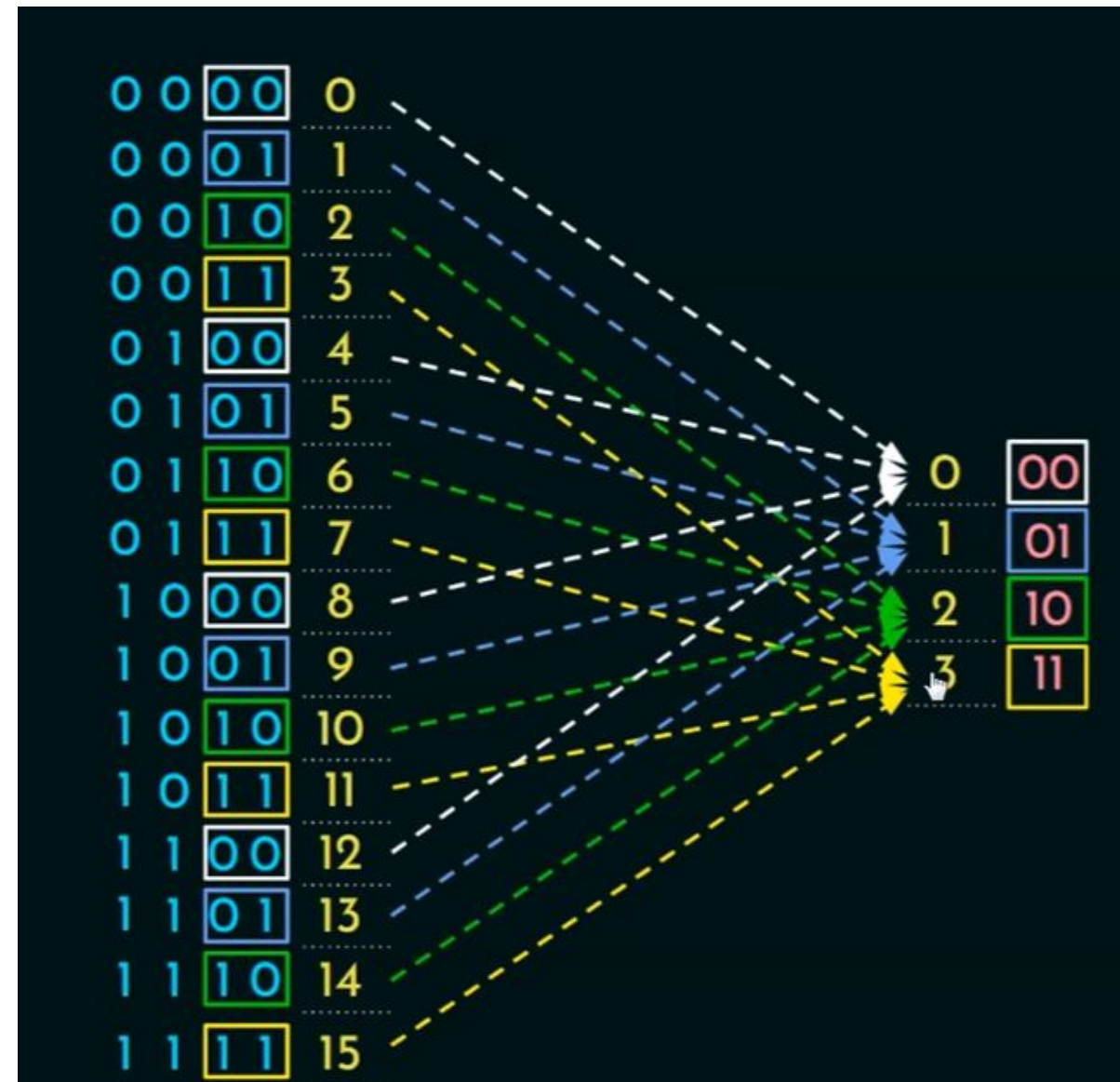
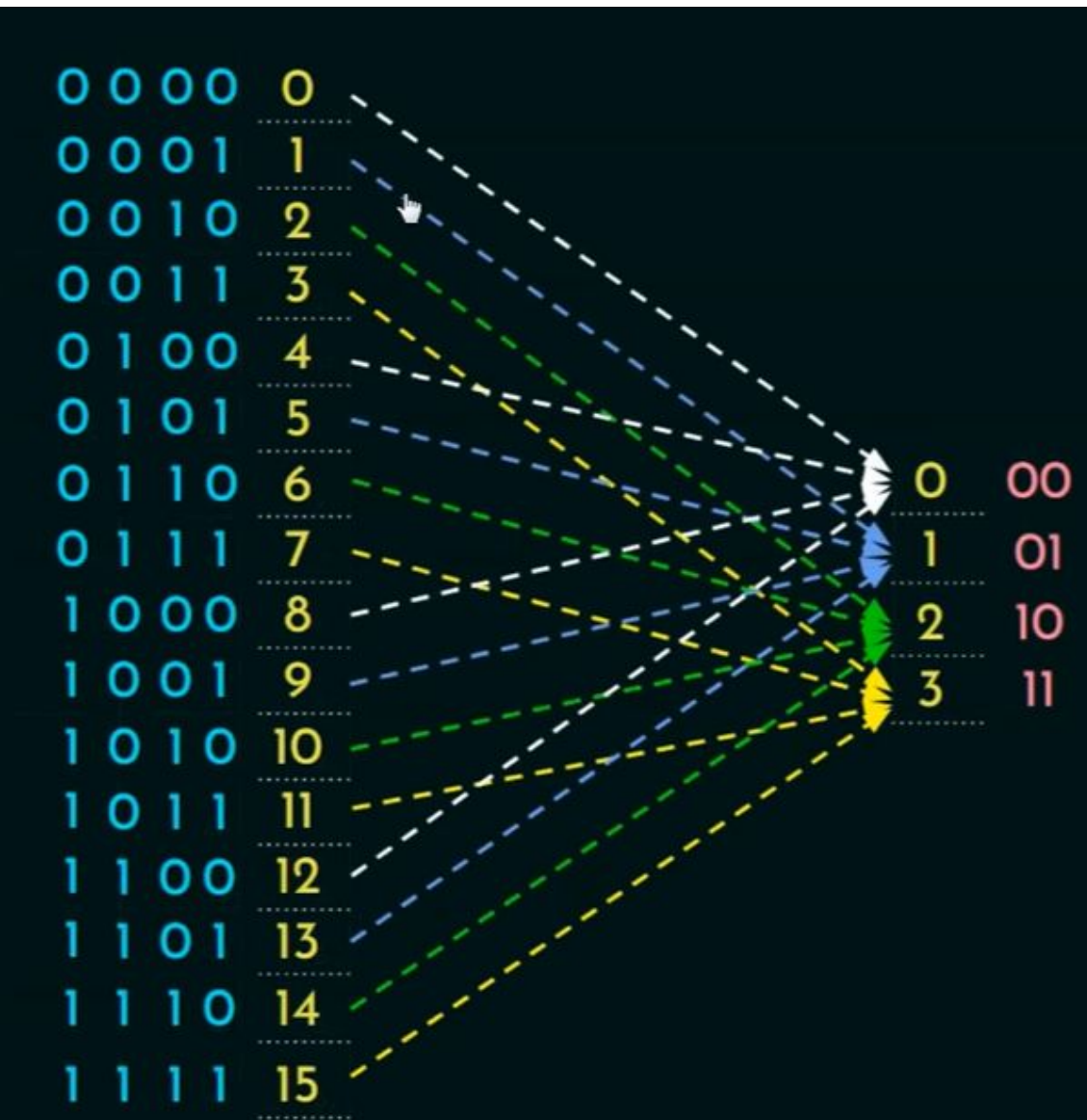
|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Main Memory to Cache Mapping





# Complete Direct Mapping: MM to Cache



# Address Structure: Direct Mapping

P. A. bits' split :

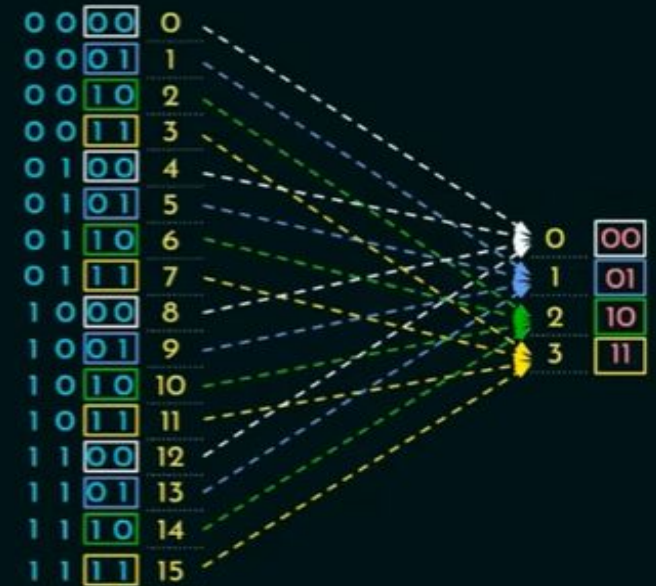
Block Number

Block / Line offset

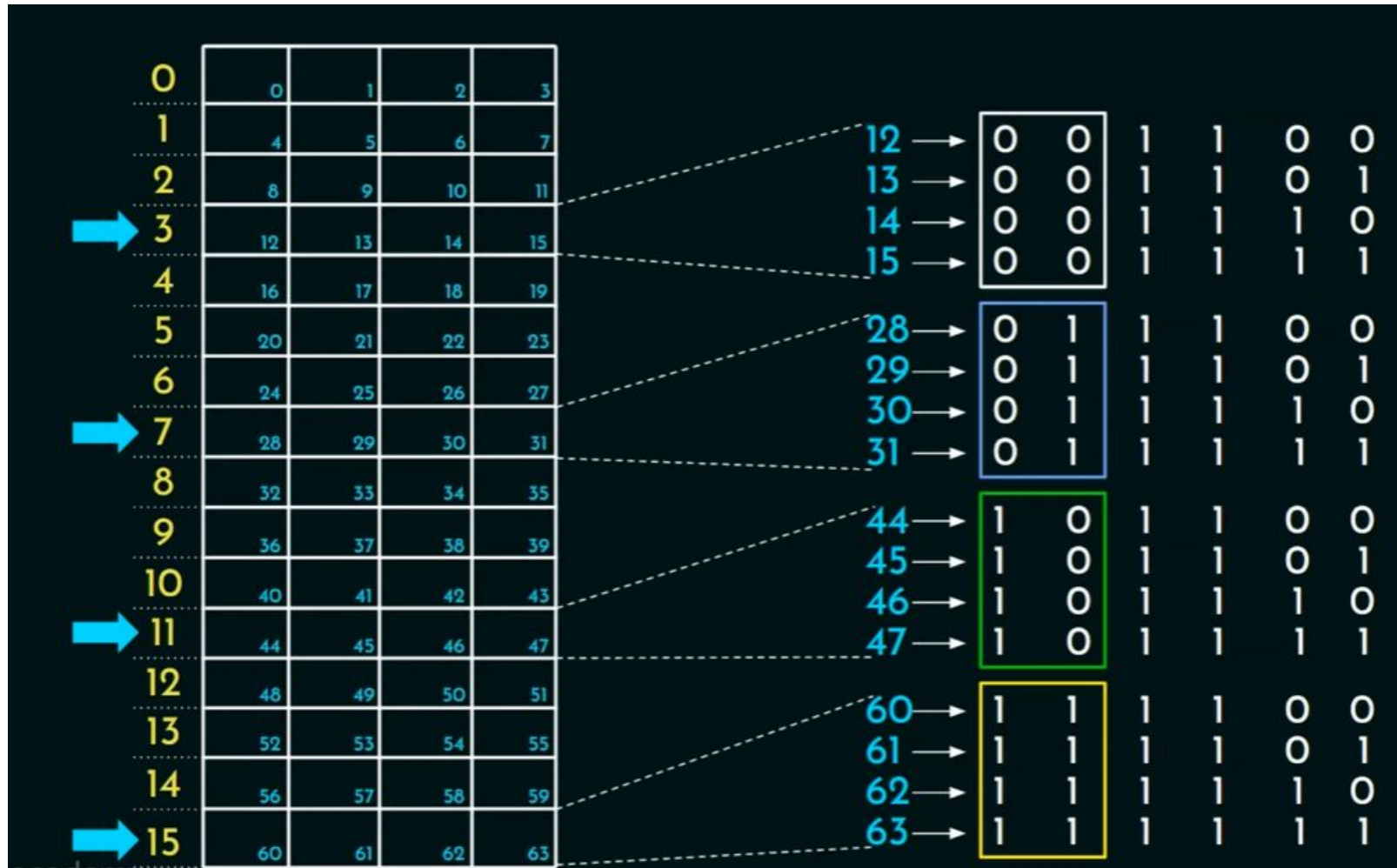
Tag bits

Line Number

Tag bits!?



# How does the mapping work?



# Let's try one example

Ex 1: MM Size: 4 GB  
Cache Size: 1 MB  
Block Size: 4 KB  
Word Size: 1 Byte

Byte addressable memory

1. P.A. bits' split?
2. Tag directory size?

Sol. MM Size = 4 GB

|        |   |                           |
|--------|---|---------------------------|
| 1 Byte | = | 8 bits                    |
| 1 KB   | = | 1024 B(ytes) = $2^{10}$ B |
| 1 MB   | = | 1024 KB = $2^{20}$ B      |
| 1 GB   | = | 1024 MB = $2^{30}$ B      |



# Solution : Physical Address Bits' Split

**Sol.** MM Size = 4 GB =  $2^2 \times 2^{30}$  B =  $2^{(2+30)}$  B =  $2^{32}$  B

$\therefore$  No. of P.A. bits =  $\log_2 2^{32} = 32$

Block Size = 4 KB =  $2^2 \times 2^{10}$  B =  $2^{12}$  B

No. of **Blocks** in MM =  $2^{32} / 2^{12} = 2^{20}$

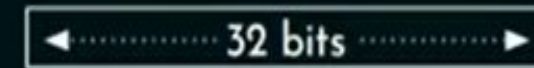
$\therefore$  **Block number** bits =  $\log_2 2^{20} = 20$

Cache Size = 1 MB =  $1 \times 2^{20}$  B =  $2^{20}$  B

No. of **Lines** in Cache =  $2^{20} / 2^{12} = 2^8$

$\therefore$  **Line number** bits =  $\log_2 2^8 = 8$

No. of **Tag** bits : P.A. bits - (Line no. bits + offset) =  $32 - (8 + 12) = 12$



$\therefore$  **Block offset** =  $\log_2 2^{12} = 12$

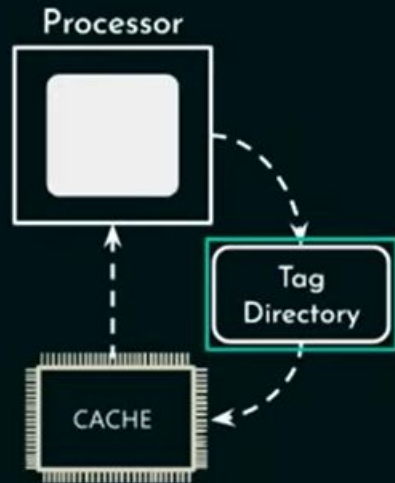




# Tag Directory Size

Ex 1: MM Size: 4 GB  
Cache Size: 1 MB  
Block Size: 4 KB

Sol.



1. P.A. bits' split?

2. Tag directory size?

Tag directory:

- Keeps primarily the record of Tag bits, cache line-wise.
- No. of entries = No. of Cache lines.

|         |        |            |
|---------|--------|------------|
| 32 bits |        |            |
| 20 bits |        |            |
| Tag     | Line   | B/L offset |
| 12 bits | 8 bits | 12 bits    |

No. of Lines in Cache =  $2^8$

No. of Tag bits = 12

Tag directory size =  $2^8 \times 12$  bits = 3072 bits

# Solve it!

## Mapping Function

- Cache of 64kByte
- Cache block of 4 bytes
  - i.e. cache is 16k ( $2^{14}$ ) lines of 4 bytes
- 16MBytes main memory
- 24 bit address
  - ( $2^{24}=16\text{M}$ )

## Direct Mapping

- Each block of main memory maps to only one cache line
  - i.e. if a block is in cache, it must be in one specific place
- Address is in two parts
- Least Significant  $w$  bits identify unique word
- Most Significant  $s$  bits specify one memory block
- The MSBs are split into a cache line field  $r$  and a tag of  $s-r$  (most significant)

# Sample Solution

## Direct Mapping Address Structure

| Tag s-r | Line or Slot r | Word w |
|---------|----------------|--------|
| 8       | 14             | 2      |

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
  - 8 bit tag (=22-14)
  - 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

# Direct Mapping Summary

- Address length =  $(s + w)$  bits
- Number of addressable units =  $2^{s+w}$  words or bytes
- Block size = line size =  $2^w$  words or bytes
- Number of blocks in main memory =  $2^{s+w}/2^w = 2^s$
- Number of lines in cache =  $m = 2^r$
- Size of tag =  $(s - r)$  bits

# Cache Hit and Cache Miss

- If the processor finds that the memory location is in the cache, a [Cache Hit](#) has occurred and data is read from the cache.
- If the processor does not find the memory location in the cache, a **cache miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from the main memory, then the request is fulfilled from the contents of the cache.
- The formula for calculating a cache hit ratio, miss ratio is as follows:

`Hit Ratio(H) = hit / (hit + miss) = no. of hits/total accesses`

`Miss Ratio = miss / (hit + miss) = no. of miss/total accesses = 1 - hit ratio(H)`

# Questions?

